Verified Projection-Event Calculus in Lean 4: Bundled Arbitrary-Partition Pinching, Lüders Retractions, and CHSH Interoperability

Bernhard Mueller*

October 6, 2026

Paper release: r2048 **Released:** October 6, 2026

Author affiliation: Bernhard Mueller, Pragma Research Inc.

Abstract

Finite quantum measurement looks simple on paper: multiply matrices, take a trace, and normalize. In Lean, each step crosses a different interface and every side condition must be supplied. This paper presents a Lean 4 library for projection events over finite complex matrix algebras. A projective partition defines a star-subalgebra of matrices that commute with its projectors and a complex-linear pinching map onto that subalgebra. The map is unital, positive, trace preserving, and idempotent; its range and fixed points are exactly the partition commutant. It also satisfies the bimodule law, Hilbert–Schmidt Pythagorean and contraction identities, and a trace-duality uniqueness theorem. It is also exactly the positive uniform random-unitary average over all independently signed block reflections. A global-sign control shows why independent signs are necessary.

A second expectation averages onto the commutative span of the partition. The two maps satisfy tower laws, preserve the partition's Born statistics, and commute with conditioning on a partition member. Lüders conditioning is available as a total matrix formula and a typed state update, with an exact fixed-state characterization. An operator-norm estimate lifts the Tsirelson theorem to a state-level Clauser–Horne–Shimony–Holt bound for projection events, and a declared maximally entangled witness on a checked slot-locality interface attains the bound $2\sqrt{2}$ exactly, while every fixed record-diagonal state read through four observables diagonal in the same basis and valued in $[-1, 1]$ obeys the classical bound 2. An ideal static phase-POVM/count model preserves the exact tomography and blindness delimitations, and an explicit synthetic inhabitant proves that exact fit alone is not an operational instrument and does not discharge the phase-sensitive-effect premise. The artifact pins its Lean and Mathlib revisions and records declaration-level axiom dependencies. Companion Kraus proofs establish complete positivity and trace preservation through a finite-matrix interface with arbitrary ancillas. A separate support countermodel proves that totalizing the matrix logarithm at zero cannot define support-aware relative entropy: the raw trace formula vanishes on an orthogonal pure-state pair.

*Corresponding author: bernhard@floatingpragma.ai

Keywords: formal verification, Lean 4, projective measurement, partition pinching, support-aware relative entropy, Lüders update, CHSH

Mathematics Subject Classification (2020): Primary 68V20; Secondary 68V15, 81P15, 81P40, 46L53.

1 Introduction

Finite quantum measurement is short on paper. A proof calls a matrix a state, inserts a projection, normalizes the result, and moves on. Lean asks for every condition skipped in that sentence. The matrix must be positive and have unit trace. The projection must be Hermitian and idempotent. Normalization needs a nonzero denominator. Later norm estimates require a further set of instances.

Machine-checked quantum information is an active area, spread over several systems and several design philosophies. Mathlib [18], the central mathematics library of Lean 4 [6], contains an order-form CHSH/Tsirelson theorem and the `IsCHSHTuple` interface [13]. Physlib's `QuantumInfo.Finite.Pinching` constructs the spectral pinching of a state as a completely positive trace-preserving map and proves commutation, fixed-point, Pythagorean, and pinching-bound results [15, 16]. Lean-Quantum provides basis-independent infrastructure for states, channels, tensor products, Choi and Kraus representations, and trace inequalities [11]; Lean-QIT develops a compositional infrastructure for quantum-information theory [22]. Zhao and Yu formalize CHSH rigidity in Lean 4 [21]. Isabelle/HOL developments cover projective measurement, quantum computation, and the CHSH upper bound [2, 7–9]. These developments choose different carriers for states and different bundling conventions, and results proved in one seldom apply verbatim in another. The engineering question underneath is interface design: which objects stay bare predicates, which get bundled, and how finished results are handed to the interfaces a large library expects. We make no priority claim for projective measurement, pinching, or Tsirelson's inequality.

The library described here packages the side conditions of finite projective measurement into a projection-event API over Mathlib matrices. Its main input is an arbitrary projective partition of the identity. The partition induces two canonical expectations: the pinching, which deletes off-block terms and maps the matrix algebra onto the partition commutant, and the average, which projects further onto the commutative span of the partition. The same event definitions feed the Lüders update and the CHSH adapter. This keeps algebraic identities separate from results that use trace, positivity, or state normalization.

The mathematics is classical: Lüders introduced the selective update rule [3, 12]; pinching by a family of orthogonal projections is a standard averaging operation in matrix analysis [1] and in quantum information [10]; the bimodule and trace-duality laws proved here are the finite-matrix forms of the defining properties of conditional expectations onto subalgebras [17, 19]; and the CHSH combination with its quantum bound goes back to Clauser, Horne, Shimony, and Holt [5] and to Tsirelson [4]. The contribution of the library is the checked interface: definitions bundled the way Mathlib expects, exact range and uniqueness theorems for both expectations, and adapters into existing Mathlib results.

Lean declaration identifiers are printed throughout in monospaced type, for example `partitionPinching_idem`; the underscore is part of the identifier. To locate a declaration, search for the exact printed identifier, after removing only TeX's protective backslash before each underscore, in the `EventAlgebra/*.lean` files. In the public repository these files are under

`Lean/EventAlgebra/`. Table 1 maps each module name to its subject matter. The top-level `EventAlgebra.lean` file imports these fifteen modules alongside the wider OPH event-algebra library.

The checked development contributes fifteen pieces:

(i) an arbitrary supplied projective partition, independent of a selected state or spectral resolution, with its commutant bundled as a `StarSubalgebra` and its span bundled with multiplication, star, commutativity, and centrality theorems;

(ii) partition pinching bundled as a complex `LinearMap`, with exact range and fixed-point theorems, positivity, unitality, trace preservation, the commutant bimodule law, Hilbert–Schmidt geometry, and map-level uniqueness;

(iii) an exact random-unitary representation of arbitrary partition pinching as the positive uniform average over all independently signed block reflections, together with a global-sign negative control;

(iv) an orthogonal-state support countermodel showing that the totalized matrix logarithm returns zero where any support-aware extended relative entropy must return infinity;

(v) partition averaging bundled as a complex `LinearMap` onto the span, with exact range, trace duality, map-level uniqueness, tower laws against the pinching, preservation of partition Born statistics, and a classical-conditioning collapse theorem;

(vi) a typed nonzero-weight Lüders state update, a support theorem and an unguarded fixed-point characterization that exhibit conditioning as a retraction onto its certainty set, and naturality of the typed update under partition pinching for commutant events;

(vii) an event-to-observable CHSH client that discharges Mathlib-compatible selfadjointness, involutivity, and cross-commutation hypotheses before applying a complementary norm-form bound;

(viii) a checked state-expectation bound that converts the norm-form theorem into a state-level CHSH bound;

(ix) a finite-state Robertson theorem for supplied density matrices and Hermitian observables, with the ordinary-commutator form, a noncommuting saturation control, and a separate zero-variance control;

(x) the exact quotient induced by a supplied projective partition: equality after block pinching is equivalent to equality under every commutant trace test, while block pinching and commutative partition averaging both erase cross-sector corners;

(xi) an exact Tsirelson saturation witness: a declared maximally entangled state and four declared projection events inhabiting a bipartite slot-locality interface of the wider library, whose CHSH expectation equals $2\sqrt{2}$ exactly, packaged with the cited upper bounds, together with a record-diagonal delimitation proving the classical bound 2 for every diagonal state read against four diagonal observables valued in $[-1, 1]$;

(xii) an ideal static phase-POVM/count-fit structure with conditional tomography, exact diagonal/real-closure blindness, and an explicit synthetic inhabitant proving that the

structure does not type source operations, producer receipts, or discharge of the phase-sensitive-effect premise;

(xiii) an exact determination boundary for that structure: a coordinate characterization of the phase-sensitivity clause, run-literal pinning of the state diagonal with one free off-diagonal coordinate, affine dependence of every count frequency on that coordinate, a decidable integer receipt window on the phase counts, and the minimal phase-mass receipt (Section 11.1);

(xiv) a static conformance layer for that structure: taking the committed exact lift as a declared phase-sensitive effect, a deterministic exact-arithmetic calculator, an independent verifier, and a composed Lean arithmetic receipt inhabit the static model with generated expected-frequency numerators satisfying the assumed Born-fit equations, the scaling law, and the integer receipt window (Section 11.2); and

(xv) a reproducible, version-pinned artifact with per-declaration axiom output, no proof placeholders, and a documented catalog of the Mathlib gaps, scoped-instance hazards, and workarounds met during construction (Section 12).

Two results are outside the current library. No theorem identifies an arbitrary partition with Physlib's spectral partition. There is no rank-one classical-representation theorem. The CHSH bounds carry a saturation witness (Section 10.2); the witness state and events are declared data, produced by no theorem from any source object. A companion module, `Dynamics/ChoiCPTP.lean`, defines an explicit finite-matrix CPTP predicate and proves both partition expectations completely positive and trace preserving; the fifteen-module audit here covers their event-algebra interfaces rather than recounting that companion development.

Section 2 fixes the mathematical and Lean interfaces. Sections 3 and 4 treat projection events and selective update. Section 5 gives the bundled partition pinching and its geometry, Section 6 the random-unitary representation and logarithm-support boundary, and Section 7 the averaging expectation, the two-level structure, and the partition-relative operational quotient. Sections 8, 9, and 10 describe the state functional, finite-state uncertainty, and the CHSH clients with the exact saturation witness and its record-diagonal delimitation. Section 11 describes the ideal static phase-POVM/count-fit model, its synthetic non-discharge control, and the declared-effect conformance fixture for the phase-count layer. Section 12 records the Mathlib engagement. Section 13 reports the audit methodology, and Section 14 positions the development feature by feature.

2 Mathematical and formal setting

Let $\mathrm{M}_n(\mathbb{C})$ be the complex $n \times n$ matrices, with conjugate transpose $X \mapsto X^*$, trace Tr, and identity $\mathbf{1}$. Following the standard finite-dimensional setting [20], a projection event is a Hermitian idempotent $P = P^* = P^2$, a state is a positive-semidefinite matrix ρ with $\mathrm{Tr}(\rho) = 1$, and the trace pairing is $\mu_\rho(M) = \mathrm{Tr}(\rho M)$.

Lean represents these interfaces as predicates over Mathlib matrices: `IsEvent P` is the conjunction $P^* = P$ and $P^2 = P$; `IsState rho` is positive semidefiniteness and unit trace. For functions that return states, the library defines the subtypes `ProjectionEvent n` and `StateMatrix n`. The split is practical. Algebraic lemmas use raw matrices. A function advertised as a state update returns a value whose type carries the state conditions.

Table 1 lists the fifteen source modules. The comparison in this work concerns their compiled interfaces and uses, not the number of declarations in each file.

Three Mathlib scopes affect compilation. `ComplexOrder` equips $\mathbb{C}$ with the partial order in which $0 \le z$ means that z is real and nonnegative. `MatrixOrder` supplies the Loewner order. `Matrix.Norms.L2Operator` activates the finite-matrix operator norm and C*-ring instances used by the matrix CHSH theorems. Without the right scope, Lean may report an instance problem even though the required theorem is in scope; Section 12 describes the failure modes.

Documentation tags classify results as *algebra-only* or *trace-dependent*. These tags do not define an import hierarchy. They record whether a proof uses ring, star, and norm structure alone, or also uses $\mathrm{Tr}(\rho M)$, positivity, or normalization.

3 Projection events and the trace pairing

The event layer proves the expected closure rules. Zero, identity, and $\mathbf{1}-P$ are events. Orthogonality is symmetric. Orthogonal sums and products of commuting events are events. Subevent absorption holds on both sides, and every event is positive semidefinite. The declaration `IsEvent.one_sub_two_smul_involution` proves that $\mathbf{1} - 2P$ is a selfadjoint involution. The CHSH adapter in Section 10 uses this result.

The trace pairing is additive, homogeneous, compatible with finite sums, and obeys the sandwich identity

$$\mathrm{Tr}(P\rho P) = \mathrm{Tr}(\rho P)$$

for idempotent P. Hermiticity of ρ and P implies that the pairing is real. Positivity of ρ and the event property imply $0 \le \mu_\rho(P)$ in Mathlib's order on $\mathbb{C}$; for a state the upper bound is $\mu_\rho(P) \le 1$. The order-form statements are strictly stronger than their real-part corollaries, and both forms are provided.

The declaration `isEvent_add_and_bornWeight_add_of_orthogonal` returns two facts: the orthogonal sum is an event, and its Born weight is the sum of the two Born weights. The event and orthogonality hypotheses carry the first conjunct; the weight identity is linear and uses none of them.

4 Lüders update as a partial state interface

For raw matrices, the library uses the totalized formula

$$\mathcal{L}_P(\rho) = \mu_\rho(P)^{-1} P\rho P.$$

Lean's field inverse maps zero to zero, so the formula is total and covers algebraic identities in the zero-weight case. Its return type is a raw matrix and carries no state guarantee. The typed operation is

$$\texttt{luedersStateUpdate} : \texttt{StateMatrix n} \to \texttt{ProjectionEvent n} \to (\mu_\rho(P) \neq 0) \to \texttt{StateMatrix n}.$$

This operation assumes `NeZero n`. The unique 0×0 matrix cannot have unit trace, so a state-returning function must exclude that dimension.

Proposition 1 (Checked Lüders interface)**.** *For a state ρ, event P, and nonzero outcome weight:*

(i) the raw formula is positive semidefinite and has trace one (`luedersUpdate_isState`);

(ii) the updated state assigns weight one to P (`bornWeight_luedersUpdate_self`);

(iii) update is an idempotent retraction (`luedersUpdate_idem`);

(iv) updates by commuting events compose and commute (`luedersUpdate_luedersUpdate_of_commute` and `luedersUpdate_comm`);

(v) when the state commutes with the event, the update is the normalized block restriction $\mu_\rho(P)^{-1}\rho P$ (`luedersUpdate_of_commute`); and

(vi) the fixed-state equivalence

$$\mathcal{L}_P(\rho) = \rho \quad \Longleftrightarrow \quad \mu_\rho(P) = 1$$

holds without a separate nonzero-weight hypothesis (`luedersUpdate_eq_self_iff`).

The fixed-point theorem needs no nonzero-weight guard. A state fixed by a zero-weight totalized update would equal the zero matrix, which has the wrong trace. The raw function covers the zero-weight case; the state theorem does not carry a redundant premise.

The module assembles these results into a fixed-point description of conditioning. The set `certainStates` collects the states that assign weight one to P. Conditioning any state of nonzero weight lands in that set in a single step (`luedersUpdate_mem_certainStates`), acts on the set as the identity, and, among states, fixes exactly its elements. Conditioning on P is therefore a retraction of the nonzero-weight states onto its own fixed-point set. The identity case rests on a support theorem, `mul_eq_self_of_bornWeight_one`: a state certain of P absorbs the event on both sides, $\sigma P = P\sigma = \sigma$. The checked proof compresses σ by the complement event, whose weight vanishes; positive semidefiniteness forces the zero-trace compression to vanish; and the Cauchy–Schwarz property of the state's quadratic form eliminates $\sigma(\mathbf{1} - P)$ itself.

5 Bundled arbitrary-partition pinching

A `ProjectivePartition n k` consists of projections $(P_i)_{i<k}$, pairwise orthogonality, and $\sum_i P_i = \mathbf{1}$. It induces

$$\mathcal{E}_\pi(X) = \sum_i P_i X P_i, \qquad N_\pi = \{X : XP_i = P_i X \text{ for every } i\}.$$

The code defines N_π as `ProjectivePartition.commutant`, a `StarSubalgebra` defined using Mathlib's centralizer, and bundles $\mathcal{E}_\pi$ as `partitionPinchingLinearMap`. The membership theorem `mem_commutant_iff` reduces membership to the pointwise commutation equation.

The range is a block-diagonal commutant and need not be commutative. For this reason, we do not call it a "classical algebra"; the classical layer is the span of Section 7. The library has no theorem identifying the span with the center of the commutant, and no rank-one classical-representation theorem.

Theorem 1 (Bundled retraction and exact range)**.** *For every projective partition π, the map $\mathcal{E}_\pi$ is complex linear, unital, positive, trace preserving, and idempotent. It lands in N_π and fixes N_π pointwise. Consequently*

$$\mathcal{E}_\pi(X) = X \iff X \in N_\pi, \qquad \mathrm{range}(\mathcal{E}_\pi) = N_\pi$$

where the second equality is the equality of the linear-map range and the underlying submodule of the bundled commutant.

The corresponding declarations are `partitionPinching_unital`, `partitionPinching_posSemidef`, `trace_partitionPinching`, `partitionPinching_idem`, `partitionPinching_eq_self_iff_mem_commutant`, and `range_partitionPinchingLinearMap`. State preservation is exposed by `partitionPinching_isState`.

Theorem 2 (Bimodule and Hilbert–Schmidt structure)**.** *If $A, B \in N_\pi$, then*

$$\mathcal{E}_\pi(AXB) = A\mathcal{E}_\pi(X)B.$$

Pinching is also selfadjoint for the trace pairing and satisfies the Pythagorean identity

$$\mathrm{Tr}(X^*X) = \mathrm{Tr}(\mathcal{E}_\pi(X)^*\mathcal{E}_\pi(X)) + \mathrm{Tr}((X - \mathcal{E}_\pi(X))^*(X - \mathcal{E}_\pi(X))),$$

and hence contracts the squared Hilbert–Schmidt quantity. A commutant-valued complex-linear map with the same trace pairings against every commutant element equals ***`partitionPinching LinearMap`****.*

The corresponding declarations are `partitionPinching_bimodule`, `trace_conjTranspose_partitionPinching_mul`, `trace_partitionPinching_pythagoras`, `trace_partitionPinching_contraction`, `trace_partitionPinching_mul_commutant`, and `partitionPinchingLinearMap_unique`. Together they make the map a positive, unital, trace-preserving linear projection with a bimodule law and a trace-duality characterization; these are the identities that characterize the trace-preserving conditional expectation onto a subalgebra in the operator-algebra literature [17, 19]. The library proves the identities directly for this map. This core module does not instantiate an operator-algebra conditional-expectation interface. The companion `Dynamics/ChoiCPTP.lean` separately proves its custom finite-matrix `IsCPTP` predicate from the same Kraus and trace identities.

5.1 Lüders naturality

If $P \in N_\pi$, compression commutes with pinching. After normalization, trace preservation against commutant elements gives

$$\mathcal{E}_\pi(\mathcal{L}_P(\rho)) = \mathcal{L}_P(\mathcal{E}_\pi(\rho)).$$

The raw formula is `partitionPinching_luedersUpdate`. The theorem `partitionPinching_luedersStateUpdate` states the same naturality for `StateMatrix n` and `ProjectionEvent n`: the input and output are states, and the nonzero-weight proof is transported by `bornWeight_partitionPinching`. This checks the same operation through the bundled state and event interfaces.

6 Random-unitary pinching and the entropy support boundary

The support-aware majorization precursor has an exact finite representation. For each sign assignment $s : \{0, \dots, k-1\} \to \{\pm 1\}$, set

$$U_s = \sum_i s(i) P_i.$$

Orthogonality and completeness make every U_s a selfadjoint unitary. The independent sign characters obey $\sum_s s(i)s(j) = 2^k \delta_{ij}$, hence

$$\frac{1}{2^k} \sum_s U_s X U_s^* = \sum_i P_i X P_i = \mathcal{E}_\pi(X).$$

The real weights are strictly positive and sum to one, so this is a literal random-unitary representation. A two-coordinate control proves that replacing the independent sign family by only the global pair $\{I, -I\}$ leaves an off-diagonal matrix unchanged and is not pinching. These statements are checked in `RecordMajorization.lean`. They are an algebraic precursor; they do not prove spectral majorization or entropy monotonicity.

The support layer cannot be bypassed by totalizing the logarithm. In Lean, $\log 0 = 0$; continuous functional calculus therefore sends the logarithm of every projection to the zero matrix, since a projection's spectrum lies in $\{0, 1\}$. Consequently the raw finite expression

$$\operatorname{Re} \operatorname{Tr}[\rho(\log \rho - \log \sigma)]$$

is zero for the two orthogonal rank-one coordinate projections. Both are density matrices, but the first support is not contained in the second. Thus any extended-real divergence that returns $+\infty$ on support failure cannot equal this totalized raw formula. The density, support, zero-value, and inequivalence receipts are checked in `SpectralEntropyBoundary.lean`.

This is a fail-closed architectural result, not a no-go for Umegaki entropy. A complete continuation must define the support-aware extended divergence and then prove the pinching relative-entropy Pythagorean identity, spectral majorization, constrained maximum-entropy formula, and the two-level publicization information chain.

7 Averaging on the partition span

The same partition carries a second canonical expectation. The *partition span*

$$D_\pi = \operatorname{span}\{P_i\}$$

is bundled as `ProjectivePartition.span`. Its closure package is proved by span induction.

Proposition 2 (The commutative layer)**.** *D_π contains every projector and the identity, and is closed under multiplication and conjugate transposition (`proj_mem_span`, `one_mem_span`, `mul_mem_span`, `conjTranspose_mem_span`). It is commutative (`span_mul_comm`). It is contained in the commutant (`span_le_commutant`), and every element of D_π commutes with every element of N_π (`mul_comm_of_mem_span_of_mem_commutant`): the span lies in the center of the commutant.*

Only the inclusion into the center is used here. The reverse inclusion is not part of this module's API; zero projectors are removed in the separate active-label coordinate equivalence. The averaging map is

$$\mathcal{A}_\pi(X) = \sum_i \frac{\mathrm{Tr}(XP_i)}{\mathrm{Tr}(P_i)}\, P_i,$$

bundled as `partitionAverageLinearMap`. Lean's zero-totalized inverse makes the terms of zero projectors vanish, so no nonzero-block hypothesis appears anywhere in the module: a zero projector contributes zero to both sides of every identity below.

Theorem 3 (Averaging expectation and exact range)**.** *For every projective partition π, the map $\mathcal{A}_\pi$ is complex linear, unital, positive, trace preserving, and idempotent. It lands in D_π, fixes D_π pointwise, and*

$$\mathcal{A}_\pi(X) = X \iff X \in D_\pi, \qquad \mathrm{range}(\mathcal{A}_\pi) = D_\pi.$$

Against every $C \in D_\pi$ the average is invisible to the trace pairing, $\mathrm{Tr}(\mathcal{A}_\pi(X)\, C) = \mathrm{Tr}(XC)$, and any D_π-valued complex-linear map with this property equals the bundled average.

The corresponding declarations are `partitionAverage_unital`, `partitionAverage_posSemidef`, `trace_partitionAverage`, `partitionAverage_idem`, `partitionAverage_eq_self_iff_mem_span`, `range_partitionAverageLinearMap`, `trace_partitionAverage_mul_of_mem`, and `partitionAverageLinearMap_unique`; state preservation is `partitionAverage_isState`. The uniqueness proof mirrors the pinching case: the difference lies in the star-closed span, is trace-orthogonal to its own conjugate transpose, and vanishes by faithfulness of the trace.

Theorem 4 (Two-level structure)**.** *The two expectations compose as a tower,*

$$\mathcal{A}_\pi \circ \mathcal{E}_\pi = \mathcal{A}_\pi = \mathcal{E}_\pi \circ \mathcal{A}_\pi,$$

(`partitionAverage_partitionPinching`, `partitionPinching_partitionAverage`). The average preserves the Born statistics of the partition, $\mu_{\mathcal{A}_\pi(\rho)}(P_j) = \mu_\rho(P_j)$ for every j (`bornWeight_partitionAverage`). For a partition member P_j of nonzero weight, averaging commutes with Lüders conditioning, and both composites collapse to the normalized projector:

$$\mathcal{A}_\pi(\mathcal{L}_{P_j}(\rho)) = \mathcal{L}_{P_j}(\mathcal{A}_\pi(\rho)) = \mathrm{Tr}(P_j)^{-1}\, P_j$$

(`partitionAverage_luedersUpdate`, `partitionAverage_luedersUpdate_proj`, `luedersUpdate_partitionAverage_proj`).

The collapse identity is the finite-matrix form of classical conditioning: on the commutative layer, conditioning on an observed partition member replaces the averaged state by the indicator of that member, normalized. The identity is stated for raw matrices; the only hypothesis is the nonzero outcome weight.

The same partition also gives an exact operational quotient. Two matrices X and Y are declared equivalent when

$$\mathrm{Tr}(XC) = \mathrm{Tr}(YC) \qquad \text{for every } C \text{ commuting with all } P_i.$$

Theorem 5 (Complete partition quotient)**.** *For every finite projective partition,*

$$\big(\forall C \in N_\pi : \mathrm{Tr}(XC) = \mathrm{Tr}(YC)\big) \quad \iff \quad \mathcal{E}_\pi(X) = \mathcal{E}_\pi(Y).$$

Consequently, every matrix D with $\mathcal{E}_\pi(D) = 0$ is invisible to all trace tests from the sector-preserving commutant. In particular, each cross-sector corner $P_i D P_j$ with $i \neq j$ lies in that kernel.

Proof. Trace duality gives the forward statistics of the pinched matrices. For the converse, apply the uniqueness theorem for the commutant-valued trace-dual projection. The cross-sector statement follows from orthogonality of the partition projectors. The complete equivalence, kernel, and corner statements are machine-checked in `Superselection.lean`. □

The theorem is relative to a supplied partition. It neither constructs a physical sector decomposition nor identifies the commutant with a laboratory readout algebra.

For every supplied projective partition, averaging maps surjectively onto the commutative partition-span algebra. It factors through block pinching, and every value commutes with the partition commutant. A one-block rank-two control keeps the maps distinct: pinching retains a within-block coherence which averaging removes. The finite matter interface bundles the same map as a typed public-record adaptor. These statements are machine-checked in `B10EdgeCenterAction.lean`. The interface adds no edge object, edge-to-partition identification, source selection, or physical detector interpretation to the event-algebra theorems.

8 The bundled state expectation

For a matrix ρ, `stateExpectationLinearMap rho` bundles $M \mapsto \mathrm{Tr}(\rho M)$ as a complex-linear functional. If ρ is a state it maps identity to one. If ρ and M are positive semidefinite, then the expectation is nonnegative. The proof of `expectation_nonneg` uses a finite spectral decomposition of M, cycles the trace, and reduces the result to a sum of products of nonnegative diagonal entries; Section 12 explains why this elementary route was chosen. Additivity and homogeneity come from the bundled `LinearMap` interface and are not repeated as pointwise lemmas.

9 Finite-state Robertson uncertainty

Let ρ be a supplied density matrix and let A and B be supplied Hermitian matrices. Write

$$A_0 = A - \mathrm{Tr}(\rho A)\mathbf{1}, \qquad B_0 = B - \mathrm{Tr}(\rho B)\mathbf{1},$$

and define

$$(\Delta_\rho A)^2 = \Re\,\mathrm{Tr}(\rho A_0^2), \qquad c_\rho(A,B) = 2\Im\,\mathrm{Tr}(\rho A_0 B_0).$$

Hermiticity makes both subtracted expectations real. The positive matrix ρ induces the seminormed pairing

$$\langle X, Y\rangle_\rho = \mathrm{Tr}(Y\rho X^*).$$

It may be degenerate when ρ is not faithful, which is why the formal proof uses Mathlib's positive-semidefinite matrix seminorm rather than silently assuming a positive-definite state.

Theorem 6 (Supplied-state Robertson inequality)**.** *For every supplied finite state and pair of Hermitian observables,*

$$\frac{c_\rho(A,B)^2}{4} \le (\Delta_\rho A)^2(\Delta_\rho B)^2.$$

The same finite pairing also gives

$$(c_\rho(A,B) : \mathbb{C}) = -i\,\mathrm{Tr}(\rho(AB - BA)).$$

In particular, the ordinary-commutator form is

$$\frac{\left|\mathrm{Tr}(\rho(AB-BA))\right|^2}{4} \le (\Delta_\rho A)^2(\Delta_\rho B)^2.$$

A zero variance on either side therefore forces the commutator readout to vanish.

Proof. Cauchy–Schwarz bounds the squared modulus of $\langle A_0, B_0\rangle_\rho$. Its imaginary part is no larger. Trace cyclicity identifies the pairing with $\mathrm{Tr}(\rho A_0 B_0)$, and scalar centring does not change the commutator. The complex identity, the centered reduction, and the ordinary-commutator inequality are machine-checked in `Robertson.lean`. □

The exact two-level control uses $\rho = |+z\rangle\langle+z|$. Pauli X and Y do not commute, have unit variance, and saturate the bound with $c_\rho(X, Y) = 2$. Pauli Z and X also do not commute, while Z has zero variance and the state commutator readout vanishes. The theorem is an uncertainty statement for supplied finite inputs. It selects no state, observable, Hamiltonian, or physical instrument.

10 CHSH interoperability as a client

For selfadjoint involutions a_0, a_1, b_0, b_1 with cross commutation, set

$$S = a_0b_0 + a_0b_1 + a_1b_0 - a_1b_1.$$

The development first proves, in a bare ring,

$$S^2 = 4\mathbf{1} - (a_0a_1 - a_1a_0)(b_0b_1 - b_1b_0)$$

(`chsh_mul_self`). In a nontrivial unital C*-ring this yields the operator-norm bound $\|S\| \le 2\sqrt{2}$ (`tsirelson_bound`), the norm form of Tsirelson's inequality [4] for the CHSH combination [5]. The analytic half is short: selfadjoint involutions have norm one by the C*-identity (`norm_eq_one_of_selfAdjoint_involution`), commutators of norm-one elements have norm at most two (`norm_commutator_le_two`), and the square identity converts these bounds into $\|S\|^2 \le 8$. The precise Mathlib structure is a `NormedRing`, `StarRing`, `CStarRing`, and `Nontrivial`. The result is stated for a unital C*-ring.

The theorem `tsirelson_bound_of_isCHSHTuple` accepts Mathlib's `IsCHSHTuple`. The matrix instantiation, `matrix_tsirelson_bound`, activates Mathlib's scoped operator norm. The declaration `matrix_tsirelson_bound_of_events` accepts four projection events plus the four cross-commutation equations, constructs the dichotomic observables $\mathbf{1}-2P$, and returns the norm bound. This gives the event layer a compiled path into the CHSH theorem.

Mathlib's `tsirelson_inequality` is an order inequality in a star-ordered real algebra [13]. The theorems in this artifact are an operator-norm bound, its state-level corollary below, and the exact equality witness of Section 10.2.

10.1 The state-level corollary

The bridge from norms to states is the checked bound

$$|\mathrm{Tr}(\rho M)| \le \|M\|$$

for every state ρ and observable M (`norm_expectation_le_l2_opNorm`). The proof diagonalizes the state, cycles the trace to $\mathrm{Tr}(D\,(V^*MV))$, dominates every diagonal entry of the conjugated observable by the operator norm (`norm_diag_entry_le_l2_opNorm`, by testing against a standard basis vector), uses that unitary conjugation does not increase the norm (`norm_eq_one_of_mem_unitaryGroup` with submultiplicativity), and sums the nonnegative eigenvalues to one. Composing with the event-level norm bound gives the state-level CHSH corollary (`matrix_state_tsirelson_bound_of_events`): for a state and four projection events with cross-party commutation,

$$\left|\mathrm{Tr}(\rho S)\right| \le 2\sqrt{2}$$

for the CHSH combination S of the dichotomic observables $\mathbf{1} - 2P$. The pairing $\mathrm{Tr}(\rho S)$ is real by `star_bornWeight`, since S is selfadjoint, so the modulus bound is a two-sided real bound. Section 10.2 exhibits a declared witness attaining the bound.

10.2 Exact saturation on the slot-locality interface

The module `TsirelsonSaturation.lean` supplies the attainment witness. Its carrier objects come from the wider library. A supplied bipartite slot split of the record algebra, committed as the interface `SlotLocalityInterface`, which carries a norm-form Tsirelson bound through the theorem `slot_locality_receipts`. The module inhabits that interface at two-dimensional slots with the singleton identity Kraus family (`bellSlotInterface`). The declared state is the normalized maximally entangled state on the four-dimensional product carrier, transported to $\mathrm{Fin}\,4$ along a fixed index equivalence so that the typed state and expectation vocabulary of Sections 2 and 8 applies (`bellState_isState`). The four declared events are the spectral projections of the two Pauli directions on the left slot and of the two quarter-rotated directions on the right slot; each carries a checked event certificate, and the four cross-party commutations are checked on the product carrier.

Theorem 7 (Exact Tsirelson saturation)**.** *The CHSH expectation of the declared state ρ_{Bell} against the combination S of the dichotomic observables $\mathbf{1} - 2P$ of the four declared events equals the upper bound exactly:*

$$\mathrm{Tr}(\rho_{\mathrm{Bell}}\, S) = 2\sqrt{2}.$$

The equality is `chsh_saturation`; its modulus form is `chsh_saturation_norm`. The packaged statement, `tsirelson_saturation_receipt`, is one conjunction over one witness: the committed norm bound of the interface cited at the witness, the state-level bound of Section 10.1 (`matrix_state_tsirelson_bound_of_events`) cited at the four events for every state on the carrier, the state certificate with the exact attainment in value and in modulus, and the identification of the attaining observable with the slot-lifted CHSH operator of the interface witness (`toFour_chshProd_eq`). The two upper bounds are cited from the committed theorems, not re-proved.

The boundary is declared data. The saturating state and the four events are declarations of the module; no theorem produces them from committed source runs, and no physical regions, spacelike separation, or observer instruments are constructed. Identification of the supplied finite event-order structure with an observer-produced physical spacetime is not supplied, and the receipt consumes the bipartite slot split as a supplied datum.

10.3 The record-diagonal classical delimitation

The same module bounds the classical side. For convex weights w and four diagonal observables a_0, a_1, b_0, b_1 valued in $[-1, 1]$ on any finite carrier, the CHSH expectation obeys the classical bound

$$\left|\sum_i w_i \left(a_{0i}b_{0i} + a_{0i}b_{1i} + a_{1i}b_{0i} - a_{1i}b_{1i}\right)\right| \le 2$$

(`diagonal_chsh_le_two`); the matrix form, for a diagonal state read against four diagonal readouts in the CHSH combination, is `diagonal_state_chsh_le_two`. In particular, the counted correlation state of the wider library, a record-diagonal state assembled from the counted joint run of the committed observer pair $(86, 247)$, obeys the bound 2 against every quadruple of diagonal $[-1, 1]$-valued readouts (`counted_state_diagonal_chsh_le_two`). Record-basis diagonality is therefore insufficient for the value $2\sqrt{2}$. This is a delimitation of one fixed diagonal state with four readouts diagonal in the same basis, not of arbitrary contextual classical experiments. The Bell state and all four attaining settings are real matrices, so the proved gap is entanglement, non-diagonal coherence, and noncommutativity; it does not isolate the genuinely complex Pauli-Y direction of the declared phase-sensitive effect, and no theorem asserts that the committed phase lift is necessary or sufficient for attainment.

The delimitation extends from one fixed counted state to the whole committed production surface (`SourceReachabilityDelimitation.lean`). The module defines the class of states reachable from the three source-counted occupation laws under every operation the development commits: the walk-step transports, the internal slot exchange, the ambient anchorings, the marginalizations, the regional and scalar conditional expectations, marginal products, and convex mixtures. Every state in that class remains diagonal in the record basis with nonnegative weights summing to one, and its CHSH value against any four record-diagonal $[-1, 1]$-valued readouts is at most 2, strictly below the attained $2\sqrt{2}$ of the declared witness. The witness state itself carries an off-diagonal entry of one half, so no map that preserves record diagonality reaches that particular witness from the class. This target-specific obstruction does not show that every realization of the value $2\sqrt{2}$ requires non-diagonal preparation: readouts that are not slot-local with respect to the declared split are outside the bound, and the slot-local separability theorem for arbitrary settings is supplied by the companion module described next. The result is a statement about the committed operation class and the declared Bell-state witness; every conceivable architecture extension is outside its scope. Like `Dynamics/ChoiCPTP.lean`, the module is a companion in the wider library, outside this paper's fifteen-module count and scale table.

The companion module `ProductSplitSeparability.lean` lifts the jointly-diagonal restriction on the declared slot split. A product-basis diagonal state with a probability diagonal is the explicit convex combination of the point states $e_{aa} \otimes e_{bb}$ weighted by its diagonal entries (`productDiagonal_separable`), and against any four Hermitian settings in the Loewner unit interval, two on each slot and with no commutation assumed inside a slot, its slot-local CHSH trace is real with modulus at most 2 (`productDiagonal_slotLocal_chsh_le_two`). Every reachable state on the two committed pair carriers is therefore separable across the split and obeys the bound against all slot-local unit-interval readouts. Slot membership is the hypothesis that carries the bound: the Bell settings conjugated by CNOT $(H \otimes 1)$ commute pairwise across the wings, lie in the unit interval, and reach exactly $2\sqrt{2}$ on the product-diagonal state $|00\rangle\langle 00|$, with one of them in neither slot (`crossWing_commutation_not_sufficient`). A slot-local value above 2 on a state certifies off-diagonal record coherence; the theorem supplies neither physical regions nor a source preparation. Like the reachability module, it is a companion in the wider library, outside the fifteen-module count and scale

table.

11 An ideal static phase-POVM count fit

The module `OperationalPhaseInstrument.lean` makes the interface boundary machine-visible. Its structure `IdealPhasePOVMCountModel` is an ideal static model: one matrix state, the committed web effects plus one designated phase effect, exact binary POVM certificates, integer count literals with the diagonal pair fixed to $(111, 68)$, and exact frequency-equals-Born equations. It contains no source operation, completely-positive outcome maps, trace-preserving summed channel, postmeasurement state, readback implementation, producer identity, receipt binding, or statistical validation rule.

Every ideal model yields fixed-trace identification (`ideal_phase_model_completes_tomography`), and its state is the unique state fitting all exact equations (`preparation_identified_by_exact_fit_data`). The construction `modelOfIdealPhaseFitData` is equivalent only to the existence of `IdealPhaseFitData` (`hasIdealPhaseFit_iff_data`); this is an algebraic fit statement rather than a theorem establishing the phase-sensitive effect.

Two incompleteness controls are exact. Two certified distinct states carry identical Born data on every diagonal effect (`diagonal_context_insufficient`), so the diagonal context alone identifies no state; the committed rotated context separates that pair; and no diagonal effect and no complexified real effect satisfies the phase-sensitivity clause (`phase_effect_not_diagonal`, `phase_effect_genuinely_complex`). Hence a complete static family needs non-diagonal content and some separator outside the complexified-real closure (`phase_context_necessary`). The theorem does not force the particular rotated effect, the designated phase effect, their order, or an operational implementation.

The non-discharge boundary is exact. The declaration `syntheticIdealPhaseFitData` chooses the rational diagonal state with weights $111/179$ and $68/179$, rotated counts $(315, 401)$, and phase counts $(1, 1)$. The theorem `synthetic_ideal_phase_fit` proves that these chosen literals inhabit `HasIdealPhaseFit` without any producer, run, operation, or receipt input. Exact finite frequency equality is also not a general validation criterion for sampled data. Therefore ideal fit cannot certify custody or establish the phase-sensitive effect. A genuine target needs source-attached CP outcome maps, a trace-preserving summed channel, operation/readback and common-preparation semantics, producer-bound receipts, and a preregistered statistical validation rule (or an explicitly exhaustive deterministic semantics). Operational additivity and composition require a separate cross-context additivity premise.

11.1 Exact determination boundary and the integer receipt window

The module `PhaseInstrumentDetermination.lean` sharpens the boundary to an exact reduction. The phase-sensitivity clause has a coordinate characterization: for a Hermitian effect E the Born-weight gap between the two Pauli-Y states equals $-2 \operatorname{Im} E_{01}$ (`bornWeight_rhoY_sub_of_isHermitian`), so the clause holds exactly when $\operatorname{Im} E_{01} \neq 0$ (`phase_sensitivity_iff_offdiag_im`). The diagonal and complexified-real exclusions are the $\operatorname{Im} E_{01} = 0$ instances, and conjugating a committed web projector by any real matrix, gauge images and permutations included, is excluded in one statement (`phase_effect_ne_real_conjugated_web`).

The committed run literals pin the model state. Every inhabitant satisfies $\rho_{00} = 111/179$, $\rho_{11} = 68/179$, and $\rho_{10} = \overline{\rho_{01}}$ (`prep_coordinates`), so one complex off-diagonal coordinate carries the entire remaining state content. Every count frequency of every inhabitant, in every instrument context, is an exact affine function of that coordinate, with real coefficients determined by the context's effect entries (`frequency_affine`). Over the committed core the phase frequency equals $1/2 - \operatorname{Im}\rho_{01}$ (`phase_frequency_eq`) and the rotated frequency equals $315/716 - (\sqrt{3}/2)\operatorname{Re}\rho_{01}$ (`rotated_frequency_eq`); two committed-core inhabitants have equal states exactly when all count frequencies agree (`prep_eq_iff_frequencies_eq`). Positivity bounds the coordinate by $|\rho_{01}|^2 \le 7548/32041$ (`prep_offdiag_normSq_le`), which forces the decidable integer window $32041\,(a-b)^2 \le 30192\,(a+b)^2$ on the phase counts (a, b) of every committed-core inhabitant (`phase_counts_receipt_window`). Both phase outcomes of every committed-core inhabitant carry positive mass, and the minimal phase count mass over committed-core inhabitants is exactly two, attained by the synthetic inhabitant (`minimal_phase_count_mass`).

The reduction states the phase residue exactly: for a committed-core inhabitant the state diagonal, the affine dependence of every count frequency on the one free coordinate, and the admissible count window are forced by the committed diagonal literals and the committed effects. Two operational requirements are not encoded by this static type: a representation of the measurement by completely positive outcome maps with a trace-preserving summed channel and compatible effects, and a source-produced common preparation that selects the context and produces the public outcomes. The window is a necessary arithmetic condition only and carries no provenance information, since the synthetic inhabitant meets it without a source run.

11.2 Static conformance fixture for the phase-effect layer

The committed exact lift $I/2 - (2\sqrt{3}/3)\,i\,(QP - PQ)$, equal to the Pauli $+Y$ projector, is taken as a declared phase-sensitive effect. It is not a state transition or an instrument. The module `OperationalPhaseAttainment.lean` consumes that declaration and inhabits `IdealPhasePOVMCountModel` over the committed core with static integer literals. A deterministic calculator expands the Born table of the declared effects on the declared matrix $\operatorname{diag}(111/179, 68/179)$ in exact arithmetic over $\mathbb{Q}(\sqrt{3}, i)$: each context pair is the exact Born weight of the context effect scaled to the least positive integer multiple of the reference mass 179 clearing its denominator, giving $(111, 68)$ at mass 179 in the diagonal and record-conjugate contexts, $(315, 401)$ at mass 716 in the rotated contexts, and $(179, 179)$ at mass 358 in the phase context. These are generated expected-frequency numerators, not observations or validation. No sampling or floating-point step occurs. An independent verifier sharing no arithmetic code rebuilds the table from the pinned payload and demands exact agreement with every receipt field, run-mass-multiple minimality and the integer window included. The composed receipt (`operationalPhaseAttainment_receipt`) derives, from one antecedent bundle pinned to the declared effect and matrix, the conjunction: committed core, declared effect in the phase slot, vanishing off-diagonal matrix entry, committed diagonal literals, the stored Born-fit equations and their cross-multiplied scaling consequences, positive integer masses, and phase counts inside the registered window. The Lean theorem does not prove leastness, source reachability of this two-dimensional matrix, or receipt provenance; the external verifier checks the deterministic least-denominator arithmetic separately. The instrument representation, the source-produced preparation, and the independent cross-context additivity premise are not supplied.

The module `LuedersPhaseInstrument` types the channel clauses as a structure `PhaseInstrument` on the eight committed contexts: outcome maps that are completely positive, trace-nonincreasing

on positive-semidefinite inputs, summing to a trace-preserving channel per context, and inducing the committed effect table. The Lüders maps $X \mapsto EXE$ inhabit it (`luedersPhaseInstrument`) with a singleton Kraus family, and the normalized post-measurement state is certain of its effect (`luedersPhaseInstrument_repeatable`). On diag$(111/179, 68/179)$ the outcome traces reproduce the fixture frequencies, with $1/2$ in the phase context, whose post-measurement state is the declared $+Y$ projector itself; the phase and diagonal values are also obtained by direct matrix computation through the outcome maps, with no use of the stored Born-fit equations. A swap-twisted instrument meets every channel clause with the same induced effects, differs on the record entry, and fails repeatability (`effect_table_does_not_determine_instrument`). Certain-state invariance does select it: a Kraus-form outcome map with projective induced effect E that leaves every state certain of E unchanged is the Lüders map $X \mapsto EXE$, and conversely, so among Kraus-form instruments with the committed effects the Lüders instrument is exactly the certain-state-invariant one; the swap-twisted instrument fails that invariance, and the committed repeatability clause (the normalized output is certain of its effect) is strictly weaker and does not select (`InstrumentSelectionByCertainStates`). Certain-state invariance is a declared operational hypothesis, no source implements it, and the instrument representation, the source-produced preparation, and cross-context additivity are not supplied. The module is a companion in the wider library, outside the fifteen-module count and scale table.

The companion module `SourcePhaseSelection` asks what phase-sensitive effect candidates can be built from source-attached algebraic fields. Its generator reads two inputs only, the six exact rows of the declared two-dimensional irreducible representation and the diagonal record projector, forms the projector orbit, and enumerates the twelve noncommuting orbit pairs. A declared complexification adapter sends each nonzero real skew commutator C to $I/2 - iC/(2|C_{01}|)$. The generated effect values are exactly the two Pauli-Y projectors (`generated_effect_values_exact`); the first pair in the current stable label order has the $+Y$ projector, which equals the declared lift and the instrument's phase slot (`current_first_labelled_effect_eq_sourcePhaseLift`, `current_first_labelled_effect_eq_declared_luedersEffect`). Eight pairs give $+Y$ and four give $-Y$, so the theorem is effect-value equality rather than witness uniqueness. Reversing the commutator order for a fixed pair swaps the transpose effects (`effectMatrix_negative_eq_transpose_positive`), but reversing the event list does not generally flip the conventional first-labelled orientation: the last current pair is also $+Y$. No theorem source-selects the first-label rule. The robust result is the unordered pair of Y projectors under the declared adapter. The module's enabled-domain relation reproduces the canonical 36/12 split between state-changing Lüders updates and equality stutters (`changingEnabledCellCensus`, `alreadyOperationEnabledCellCensus`). It constructs no instrument and supplies no preparation, outcome, readback, or provenance; the phase-sensitive-effect declaration keeps the adapter and orientation choice. Like the reachability module, it is a companion in the wider library, outside the fifteen-module count and scale table.

The companion module `SourcePhaseInstrumentOutcomeBridge` joins those generated source semantics to the declared Lüders phase instrument. It maps every generated $+Y$ event to the declared phase-outcome index 0 and every generated $-Y$ event to index 1, then proves exact equality of the generated effect with that entry of the committed effect pair (`generatedEffect_eq_committedPhaseOutcome`). All five generated state matrices are positive semidefinite with trace one (`stateMatrix_isState`). For each of the 48 enabled source steps the corresponding Born weight is nonzero, and the normalized output of the indexed declared Lüders map equals the generated semantic result matrix (`sourceStep_normalized_luedersOutcome_eq_result`). This is a matrix-level compatibility theorem. The index is not a recorded public outcome, and the result neither source-selects nor implements the Lüders instrument, nor supplies one common cross-

context preparation, readback, run binding, provenance, or custody. The clauses are conjoined in `sourcePhaseInstrumentOutcomeBridge_receipt`.

The companion module `SourcePhaseCommonPreparationHull` identifies the exact common-support surface of the generated phase relation. Exactly the three real orbit states admit all twelve generated phase events (`hasAllPhaseEvents_iff`). Their normalized real mixture with weights

$$\frac{265}{537}, \qquad \frac{136}{537}, \qquad \frac{136}{537}$$

is exactly the declared run state; this is theorem `commonSupportMixture_eq_committedRunState`. Equality to that matrix uniquely determines all three coefficients, without assuming normalization (`commonSupportMixture_coefficients_unique`). Each common-support state assigns weight $1/2$ to either generated Pauli-Y orientation, and every common-support state/event pair inherits the normalized declared Lüders-outcome equality. The declared run matrix is not any generated state, no generated source step starts from it, and the result of every generated phase event lacks all-event support. This is retrospective convex-hull compatibility, not a source-produced or enabled common preparation. The pointwise one-step cells supply no convex-lift, mixing, reset, re-preparation, or sequential schedule. The twelve generated events are not the eight public instrument contexts, and the theorem supplies no instrument implementation, public outcome, readback, run binding, provenance, custody, cross-context additivity, or physical attachment. The clauses are conjoined in `sourcePhaseCommonPreparationHull_receipt`.

The module `SourcePhaseBornWeightBoundary` proves the exact single-shot identity

$$\mathrm{Tr}(\rho P_{+Y}) = \frac{1}{2} - \mathrm{Im}(\rho_{01})$$

for every normalized two-dimensional state. Thus a non-half weight for the current conventional first-labelled $+Y$ effect under the declared adapter/order weight requires nonzero imaginary off-diagonal coherence; real off-diagonal coherence alone is insufficient (`state_current_first_labelled_weight_eq_half_sub_im`). It then places that conventional effect and the enumerated source-reachable class behind an explicit prospective preparation-bridge interface. If such a bridge maps reachable matrices to states and preserves record diagonality, the selected $+Y$ effect has Born weight exactly $1/2$ on every bridged preparation (`reachable_recordPreserving_phase_weight_half`). On a record-diagonal state the generated $-Y$ orientation and the diagonal comparator $(1/2)I$ have the same weight. Constant diagonal and off-diagonal bridges show that the preservation hypothesis is logically inhabitable and load-bearing. Both are mathematical controls that ignore their inputs. The result derives no source bridge or operation, common preparation, public outcome, sequential instrument behavior, operational readback, run, provenance, or custody.

12 Working over Mathlib: scopes, gaps, and affordances

This section records what the construction consumed from Mathlib, where the library resisted, and which workarounds closed the gaps. The artifact repeats the catalog in the notes file `MATHLIB_NOTES.md`.

12.1 Scoped instances

The three scopes of Section 2 share a failure mode. The partial order on $\mathbb{C}$ and the Loewner order are scoped instances; every $0 \le z$ goal and every positive-semidefiniteness statement over $\mathbb{C}$

elaborates only when the scope is open, and a missing scope surfaces as an instance-resolution error far from its cause. `ComplexOrder` also carries the scoped ordered-ring structure on $\mathbb{C}$ that supplies multiplicativity of nonnegativity inside the complex order. There is no global norm on Mathlib matrices; the operator norm, the `NormedRing` structure, and the `CStarRing` instance arrive only under `Matrix.Norms.L2Operator`, so the entire Tsirelson section lives behind that scope. The `Nontrivial` instance for $n \times n$ complex matrices with $n \neq 0$ did not synthesize and is constructed by hand, entrywise. The matrix unitary group is a submonoid distinct from Mathlib's `unitary` bundle, so its norm-one property is also proved by hand from the C*-identity.

12.2 Gaps and workarounds

Bilinear trace positivity. Mathlib proves $0 \leq \mathrm{Tr}(A)$ for a single positive-semidefinite matrix and covers sandwich forms BAB^*. The bilinear statement $0 \leq \mathrm{Tr}(AB)$ for two positive-semidefinite factors is absent. For Born weights the gap costs nothing, because $\mathrm{Tr}(\rho P) = \mathrm{Tr}(P\rho P)$ for idempotent P. For `expectation_nonneg` the natural route, the C*-order equivalence between nonnegativity and Gram factorization, fails to elaborate: two continuous-functional-calculus instances are not synthesized for the matrix algebra under its product topology. The proof takes the elementary route instead, a spectral decomposition of the observable followed by a trace cycle and termwise nonnegativity, in about a dozen lines.

The ring identity. The square identity behind the Tsirelson bound is pure noncommutative ring algebra under side relations: four involutivity equations and four cross commutations. Lean has no noncommutative analogue of `linear_combination`, and `noncomm_ring` does not use hypotheses. The checked proof builds a small confluent rewrite system. Each commutation hypothesis is recast as a universally quantified, association-compatible form $b(ax) = a(bx)$, and each involution as $a(ax) = x$; applied through `simp only`, these rules move every b past every a in right-associated words and cancel adjacent equal letters. Every rewrite strictly decreases the number of letter pairs out of normal order, so the system terminates, and `abel` closes the remaining additive goal.

Higher-order motives. Two steps needed their motives spelled out. The binary span-induction principle behind the closure package of Proposition 2 does not infer its motive from the goal; the predicate is passed explicitly. Rewriting with the unit-trace equation inside a hypothesis whose proof term mentions the bundled state predicate fails the motive typecheck, because the state predicate itself contains the trace being abstracted; a term-level chain through injectivity of the real embedding replaces the rewrite.

Rewriting discipline. Unfolding the definition of the Born weight rewrites the first syntactic occurrence, which in normalized expressions is often the normalizing scalar $\mu_\rho(P)^{-1}$ instead of the intended trace argument. The affected proofs isolate each trace identity as a standalone step with explicit arguments to the trace commutation and cycling lemmas.

12.3 Affordances

The `Matrix.PosSemidef` API is complete enough that the development never unfolds the definition of positive semidefiniteness: sandwich closure, sums, scalar multiples, diagonal nonnegativity, trace nonnegativity, and the Gram construction cover every need, and the scalar lemma accepts a complex scalar that is nonnegative in the complex order, which makes the Lüders normalization a single application. Trace faithfulness is available in two forms: the conjugate-transpose criterion powers the

uniqueness theorems of both expectations, and the zero-trace criterion together with the Cauchy–Schwarz property of positive matrices powers the support theorem of Section 4. On the analytic side, the C*-identity, the norm-one property of the unit, and submultiplicativity reduce the norm half of the Tsirelson bound to about forty lines over the four-typeclass signature, and the matrix case is a two-line instantiation behind the norm scope. The state-expectation bound of Section 10.1 rests on the scoped norm being definitionally the Euclidean operator norm: the type synonym is identity-transparent, so a `show` converts Euclidean-space statements to raw matrix-vector algebra, and the basis-vector, submultiplicativity, and eigenvalue-trace lemmas finish the proof in about sixty lines. Mathlib's CHSH file supplied the `IsCHSHTuple` bundle, so interoperability with the order-form theorem cost one conversion theorem.

13 Artifact evaluation and proof audit

The source is a Lake project. Its toolchain pins `leanprover/lean4:v4.29.1`; its manifest pins Mathlib commit `5e932f97dd25535344f80f9dd8da3aab83df0fe6`. Build the library with

```
lake build EventAlgebra.
```

The canonical Lean modules are publicly available in the Observer Patch Holography repository [14]. The exact source snapshot is the repository revision containing this manuscript and its built PDF; the manuscript source is in the repository.

Table 2 describes the fifteen modules discussed here and the umbrella root. Other modules imported by the broader root are outside this paper's count. The comparison in Section 14 concerns interfaces, not counts.

Every public result discussed in the paper appears under its Lean name. The modules end with `#print axioms` commands for the principal declarations, and a clean build emits the generated output. The 277 audited declarations report only subsets of propositional extensionality, classical choice, and quotient soundness, the standard Lean/Mathlib base. No declaration reports `sorryAx`, and a source scan finds no `sorry` placeholder in these modules.

The API uses every orthogonality hypothesis in its orthogonal-sum theorem. The sequential-update theorem has no unused event premise. State update is typed and excludes dimension zero. The fixed-point equivalence has no nonzero-weight guard, and the averaging module carries no nonzero-block hypotheses. The commutant, span, and both expectations are bundled, and the CHSH adapters are clients of the event definitions.

The repository contains the toolchain, Lake manifest, source modules, build command, in-source axiom queries, and a Creative Commons BY-NC-SA 4.0 license. No persistent archive identifier is claimed.

14 Feature-level comparison

Table 3 compares compiled features. It does not treat a count of elementary lemmas as a research contribution.

Physlib provides more channel structure: its spectral pinching is a `CPTPMap` [15]. This artifact

accepts a supplied partition that need not come from the spectrum of a state. The two APIs therefore have different inputs, and no correspondence theorem connects them.

Lean-Quantum and Lean-QIT cover a larger part of quantum information [11, 22]. Here, the scope is limited to connections among projections, commutants, spans, the two expectations, selective update, and Mathlib's CHSH interface.

15 Limitations and conclusion

The library connects projection events, Born weights, a typed Lüders update, arbitrary projective partitions, the two expectations of a partition, and event-level CHSH bounds in norm and state form, with an exact saturation witness and a record-diagonal classical delimitation on the same event API. The pinching has the bundled commutant as its exact range and the average has the bundled span; each map is the unique trace-dual projection onto its range, the two compose as a tower, and both interact with Lüders conditioning by checked theorems. The supplied-state Robertson bound, the independent-sign representation, the logarithm-support boundary, and the partition-relative operational quotient use the same finite matrix API. Each formal statement in the paper points to a declaration in the repository source.

The construction follows one pattern: state algebraic content as predicates over raw matrices, move to subtypes exactly where an API promises a state, bundle the carrier and the projection so that range and uniqueness theorems are exact, and hand finished objects to Mathlib interfaces through small adapters. The pattern is independent of measurement theory and applies to other finite-dimensional operator-algebra developments over Mathlib.

The companion `Dynamics/ChoiCPTP.lean` establishes complete positivity and trace preservation for both expectations under a custom finite-matrix `IsCPTP` interface. There is no theorem connecting the supplied partition to Physlib's spectral pinching and no rank-one classical specialization. The CHSH saturation witness rests on declared data: no theorem produces the state or the events from a committed source run. The ideal static phase-fit model has an explicit synthetic inhabitant and is not an operational instrument. The entire development is finite-dimensional and says nothing about normal maps on infinite-dimensional von Neumann algebras.

The proved surface contains the arbitrary-partition pinching with its exact range and bimodule law, the averaging expectation with its tower and classical-conditioning laws, typed Lüders naturality, the complete partition-relative quotient, the supplied-state Robertson inequality, the state-level CHSH corollary, the exact saturation receipt with its record-diagonal delimitation, and the ideal phase-fit model with its synthetic non-discharge counterreceipt; the companion module adds the custom CPTP proofs. It contains no Physlib spectral-partition correspondence and no source-selected quantum instrument; the two-qubit saturation witness consists of declared data, and the operational phase instrument and its source custody are not constructed.

Declarations

Funding. No funding was received for this work.

Competing interests. The author declares no competing interests.

Author contributions. Bernhard Mueller is the sole author, designed the study, reviewed the

formal development, and takes responsibility for the manuscript and artifact.

Data and code availability. No empirical data were generated. The canonical Lean source is available in the Observer Patch Holography repository [14]. The repository revision containing this manuscript provides the Lean source, pinned dependency files, build instructions, and declaration-level axiom queries under a Creative Commons BY-NC-SA 4.0 license. No persistent archive identifier is claimed.

References

[1] Rajendra Bhatia. Pinching, trimming, truncating, and averaging of matrices. *The American Mathematical Monthly*, 107(7):602–608, 2000.

[2] Anthony Bordg, Hanna Lachnitt, and Yijun He. Certified quantum computation in Isabelle/HOL. *Journal of Automated Reasoning*, 65:691–709, 2021. https://doi.org/10.1007/s10817-020-09584-7.

[3] Paul Busch and Pekka Lahti. Lüders rule. In Daniel Greenberger, Klaus Hentschel, and Friedel Weinert, editors, *Compendium of Quantum Physics*. Springer, Berlin, Heidelberg, 2009.

[4] Boris S. Cirel'son. Quantum generalizations of Bell's inequality. *Letters in Mathematical Physics*, 4(2):93–100, 1980. https://doi.org/10.1007/BF00417500.

[5] John F. Clauser, Michael A. Horne, Abner Shimony, and Richard A. Holt. Proposed experiment to test local hidden-variable theories. *Physical Review Letters*, 23(15):880–884, 1969. https://doi.org/10.1103/PhysRevLett.23.880.

[6] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. https://doi.org/10.1007/978-3-030-79876-5_37.

[7] Mnacho Echenim. Quantum projective measurements and the CHSH inequality. *Archive of Formal Proofs*, 2021. Formal proof development, ISSN 2150-914x.

[8] Mnacho Echenim and Mehdi Mhalla. A formalization of the CHSH inequality and Tsirelson's upper-bound in Isabelle/HOL. *Journal of Automated Reasoning*, 68:Article 2, 2024. https://doi.org/10.1007/s10817-023-09689-9.

[9] Mnacho Echenim, Mehdi Mhalla, and Coraline Mori. The CHSH inequality: Tsirelson's upper-bound and other results. *Archive of Formal Proofs*, 2023. Formal proof development, ISSN 2150-914x.

[10] Masahito Hayashi. *Quantum Information Theory: Mathematical Foundation*. Graduate Texts in Physics. Springer, Berlin, Heidelberg, 2nd edition, 2017. https://doi.org/10.1007/978-3-662-49725-8.

[11] Kazumi Kasaura, Kei Tsukamoto, Kento Mori, Risa Mizuno, Takahiro Namatame, Yuta Oriike, Masaya Taniguchi, Sho Sonoda, and Hayata Yamasaki. Lean-Quantum: Toward AI-assisted formalization of quantum information, 2026. arXiv:2607.05492, https://doi.org/10.48550/arXiv.2607.05492.

[12] Gerhart Lüders. Über die zustandsänderung durch den meßprozeß. *Annalen der Physik*, 8: 322–328, 1951. English translation by K. A. Kirkpatrick, *Concerning the state-change due to the measurement process*, Annalen der Physik **15**(9), 663–670 (2006).

[13] Kim Morrison and Mathlib contributors. Mathlib.algebra.star.chsh. Mathlib documentation, 2026. URL https://leanprover-community.github.io/mathlib4_docs/Mathlib/Algebra/Star/CHSH.html. Accessed 20 July 2026.

[14] Observer Patch Holography contributors. Observer Patch Holography: Source repository. https://github.com/FloatingPragma/observer-patch-holography, 2026. Commit 49ee36e01eb245d4d508e82c9b08fd47f7a85243; accessed 21 July 2026.

[15] Physlib contributors. Quantuminfo.finite.pinching: Pinching channels. Lean library documentation, 2026. URL https://ohaithe.re/Lean-QuantumInfo/QuantumInfo/Finite/Pinching.html. Accessed 20 July 2026.

[16] Physlib/QuantumInfo contributors. QuantumInfo: A Lean 4 library for quantum information, 2026. Software project; finite mixed states as positive-semidefinite trace-one matrices, POVMs, and Born-rule measurement.

[17] Masamichi Takesaki. Conditional expectations in von Neumann algebras. *Journal of Functional Analysis*, 9(3):306–321, 1972. https://doi.org/10.1016/0022-1236(72)90004-3.

[18] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. https://doi.org/10.1145/3372885.3373824.

[19] Hisaharu Umegaki. Conditional expectation in an operator algebra. *Tôhoku Mathematical Journal*, 6(2–3):177–181, 1954. https://doi.org/10.2748/tmj/1178245177.

[20] John von Neumann. *Mathematische Grundlagen der Quantenmechanik*. Springer, Berlin, 1932. English translation: *Mathematical Foundations of Quantum Mechanics*, Princeton University Press, Princeton, 1955.

[21] Tianrun Zhao and Nengkun Yu. Formalizing CHSH rigidity in Lean 4, 2026. arXiv:2604.03884, https://doi.org/10.48550/arXiv.2604.03884.

[22] Chengkai Zhu, Ziao Tang, Guocheng Zhen, Yimeng Cao, Yusheng Zhao, Ranyiliu Chen, Xuanqiang Zhao, Lei Zhang, and Xin Wang. Lean-QIT: Towards a formal infrastructure for quantum information theory, 2026. arXiv:2607.09632, https://doi.org/10.48550/arXiv.2607.09632.

AI Assistance Disclosure

This research project used research-grade commercial models, including Anthropic's Fable and OpenAI's GPT-6 Astra, for research support, software development, editing, and synthesis. The authors are responsible for the paper's claims, methods, and final text.

Module	Principal interface
`Basic`	projection events, states, trace pairing, closure and Born weight bounds
`Lueders`	totalized matrix formula, typed state update, repeatability, composition, support, and fixed points
`Partition Pinching`	projective partitions, bundled commutant, bundled linear pinching, exact range, geometry, bimodule law, and naturality
`PartitionAverage`	partition span with closure, commutativity, and centrality; bundled averaging expectation with exact range, uniqueness, tower laws, statistics preservation, and classical conditioning
`StateExpectation`	trace pairing bundled as a complex-linear functional, normalization and positivity
`Robertson`	supplied-state Robertson inequality, ordinary-commutator identity, variance nonnegativity, and exact Pauli controls
`Superselection`	complete operational quotient for partition pinching and cross-sector invisibility for both declared readouts
`Record Majorization`	positive uniform independent-sign random-unitary representation of arbitrary partition pinching and a global-sign negative control
`SpectralEntropy Boundary`	exact support-failure density witness and totalized-matrix-logarithm no-go for support-aware relative entropy
`Tsirelson`	CHSH ring identity, norm theorem, Mathlib interoperability, and event-level matrix client
`ExpectationBound`	state-expectation operator-norm bound and the state-level CHSH corollary
`Tsirelson Saturation`	declared Bell witness on the slot-locality interface attaining $2\sqrt{2}$ exactly, packaged saturation receipt, and record-diagonal classical delimitation
`OperationalPhase Instrument`	ideal static phase-POVM/count-fit structure, conditional tomography, exact blindness receipts, and a synthetic non-discharge counterreceipt
`PhaseInstrument Determination`	coordinate characterization of the phase clause, run-literal state pinning, affine count-frequency dependence, decidable integer receipt window, and the minimal phase-mass receipt
`OperationalPhase Attainment`	declared-effect inhabitant of the count-fit structure with generated expected-frequency numerators, stored Born-fit equations, scaling consequences, and the receipt-window clause

Table 1: Module structure of the checked development.

Module	Lines	Audited declarations
`Basic`	302	25
`Lueders`	294	14
`PartitionPinching`	474	27
`PartitionAverage`	513	30
`StateExpectation`	105	4
`Robertson`	416	20
`Superselection`	197	9
`RecordMajorization`	322	6
`SpectralEntropyBoundary`	161	5
`Tsirelson`	275	8
`ExpectationBound`	174	4
`TsirelsonSaturation`	834	39
`OperationalPhaseInstrument`	878	40
`PhaseInstrumentDetermination`	504	24
`OperationalPhaseAttainment`	373	22
`EventAlgebra` (root)	172	0
Total	5994	277

Table 2: Source lines and per-module counts of declarations covered by the generated axiom audit.

Feature	Closest checked infrastructure	This artifact
Projective measurement	Isabelle/HOL projective-measurement developments; state/measurement interfaces in current Lean libraries	finite matrix event predicate, typed event/state subtypes, closure and trace-pairing lemmas; treated as prerequisite; no priority claim
Pinching	Physlib spectral pinching is a state-selected bundled CPTP map with fixed-point and geometric results	arbitrary supplied projective partition; bundled linear map and commutant; companion custom-`IsCPTP` proof; no Physlib correspondence theorem
Retraction structure	operator-algebra conditional expectations and channel interfaces	exact range/fixed points, positive/unital/trace-preserving, bimodule, Hilbert–Schmidt geometry, map-level trace-duality uniqueness
Random-unitary and entropy boundary	random-unitary dephasing and support-aware Umegaki divergence in finite quantum information	exact independent-sign average for arbitrary supplied partitions; a support-failure countermodel showing that totalized matrix logarithms cannot encode the extended divergence
Commutative layer	conditional expectations onto commutative subalgebras in operator-algebra practice	bundled averaging expectation onto the partition span, exact range and uniqueness, tower laws with pinching, Born-statistics preservation, classical-conditioning collapse
Lüders update	Isabelle projective measurement and general channel infrastructure	raw formula plus typed partial state API, support theorem, unguarded state fixed-point equivalence, typed naturality with arbitrary-partition pinching
CHSH/ Tsirelson	Mathlib order theorem; Isabelle upper bound; Lean CHSH rigidity	complementary norm theorem, event-level client, state-expectation corollary, and an exact saturation witness on declared data; fixed record-diagonal states with jointly diagonal readouts capped at the classical bound
Reproducibility	public CI and archived releases are common best practice	pinned Lean/Mathlib, an exact repository snapshot, clean build instructions, and declaration-level axiom queries; no DOI claim

Table 3: Direct comparison with the closest formal infrastructure.